CNN Image Classification

NAME: NANDITA R NADIG
ID: PES2UG23CS365
COURSE NAME: MACHINE- LEARNING (UE23CS352A)
SUBMISSION DATE: 20.11.2025

Introduction

In this lab, the goal is to build, train, and test a Convolutional Neural Network (CNN) that can classify images of hands showing Rock, Paper, or Scissors. We start by preprocessing a real-world dataset, then design the CNN from scratch, and train it using PyTorch. Once the model is trained, we test how well it performs, both on individual images and in a simulated Rock–Paper–Scissors game.

This lab is a great way to dive into concepts like image preprocessing, convolutional layers, pooling, and fully connected networks. The whole process helps me understand how these components work together to create a functional model and evaluate its performance.

Model Architecture

This CNN model consists of three convolutional blocks, each designed to progressively extract more complex features from the input images:

Block 1:

- The first layer is a convolutional layer that takes in the image and applies filters to create feature maps. It outputs 16 feature maps and uses a kernel size of 3x3 with padding to keep the image size the same.
- After the convolution, a ReLU activation is applied to introduce non-linearity, followed by a MaxPooling layer that reduces the image size by half.

Block 2:

- The second block does something similar, but this time it increases the number of output feature maps to 32.
- Again, apply ReLU activation and MaxPooling to reduce the image size.

Block 3:

- The final block increases the output feature maps to 64.
- Once again, ReLU activation is applied, followed by MaxPooling, further downsampling the image.

After these three blocks, the original image has been reduced to a much smaller size, but with richer, more abstract features.

For the fully connected part:

- A Flatten layer converts the 3D feature maps into a 1D vector.
- A Linear layer reduces the vector to a smaller set of features, followed by another ReLU activation.
- A Dropout layer is used to randomly disable some connections during training, which helps prevent overfitting.
- The final Linear layer outputs predictions corresponding to the target classes.

Training and Performance

The model was trained with the following hyperparameters:

- Optimizer: Adam

- Loss Function: Cross Entropy Loss
- Learning Rate: 0.001
- Epochs: 10
- Batch Size: 32

After 10 epochs of training, the model achieved 97.03% accuracy.

Conclusion and Analysis

The model worked well, achieving an accuracy of 97%. This shows that the CNN was able to learn useful features for distinguishing between the three hand gestures. The steady drop in training loss and the solid performance on the test set suggest that the model generalized well to new, unseen images.

Challenges faced:

- Consistent preprocessing: Making sure the preprocessing was the same for both training and inference was a bit tricky.
- Dataset organization: Managing file paths and the structure of the dataset posed some challenges, especially when dealing with a large number of images.
- Model depth vs. overfitting: It was important to ensure the model was deep enough to capture complex patterns but not so deep that it overfitted to the training data.

Possible improvements:

- Data Augmentation: Adding techniques like *RandomRotation*, *ColorJitter*, or *HorizontalFlip* could help make the model more robust to variations in the input images.
- Transfer Learning: Using a pretrained model like *ResNet18* could boost accuracy even more and reduce the training time since it would leverage features learned from a much larger dataset.